SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week6 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
NAME	B.SUSHMA	**BATCH**	48
Assignment Number: 11.1(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 11 – Data Structures with AI: Implementing Fundamental Structures** **Lab Objectives** • Use AI to assist in designing and implementing fundamental data	Week6 - Monday

structures in Python.

- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
    pass
```

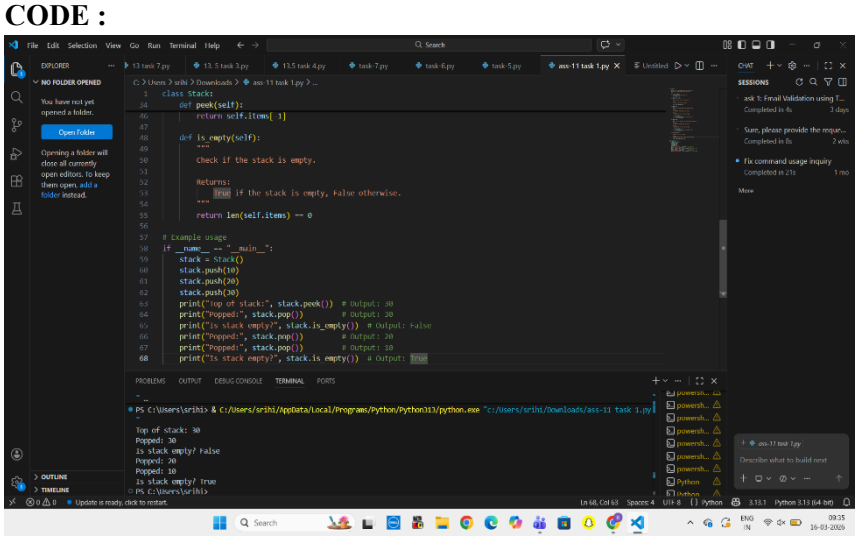
PROMPT : Generate a Python class named Stack that implements a stack data structure using a list.

Include the following methods: push(), pop(), peek(), and is_empty().

Follow the LIFO principle and add proper comments and docstrings explaining each method.

Also include a small example usage.

CODE :



Expected Output:

- A functional stack implementation with all required methods and docstrings.

Explanation : In this task, I implemented the Stack data structure using Python. A stack follows the LIFO (Last In First Out) principle, which means the element that is inserted last will be removed first.First, I opened Visual Studio Code and created a new Python file. Then I defined a class called Stack. Inside the class, I created a list to store the stack elements.After that, I implemented the main

stack operations such as to insert an element, to remove the top element, to view the top element, to check whether the stack is empty. Finally, I tested the stack operations by inserting and removing elements.

Task Description #2 – Queue Implementation

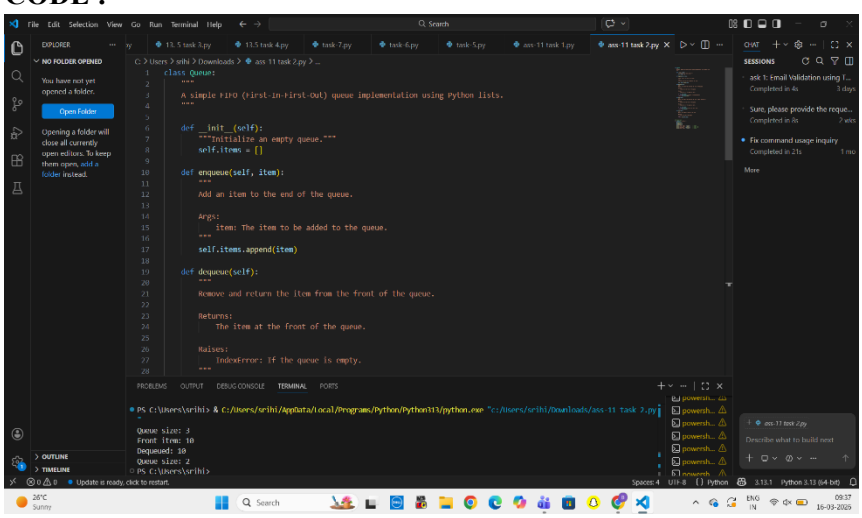
Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
    pass
```

PROMPT : Write a Python class Queue that implements a FIFO queue using Python lists.
Include the methods enqueue(), dequeue(), peek(), and size().
Add comments and docstrings explaining the functionality of each method
and provide a short example demonstrating queue operations.

CODE :



Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Explanation : In this task, I implemented the Queue data structure using Python lists. A queue follows the FIFO (First In First Out) principle, which means the first element inserted will be removed first.

I created a class named in a Python file. Inside the class, I used a list to store queue elements. Then I implemented methods such as to add elements to the queue to remove elements from the queue.

I also implemented to check the first element and to determine the number of elements in the queue. Finally, I executed the program to verify the working of queue operations.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

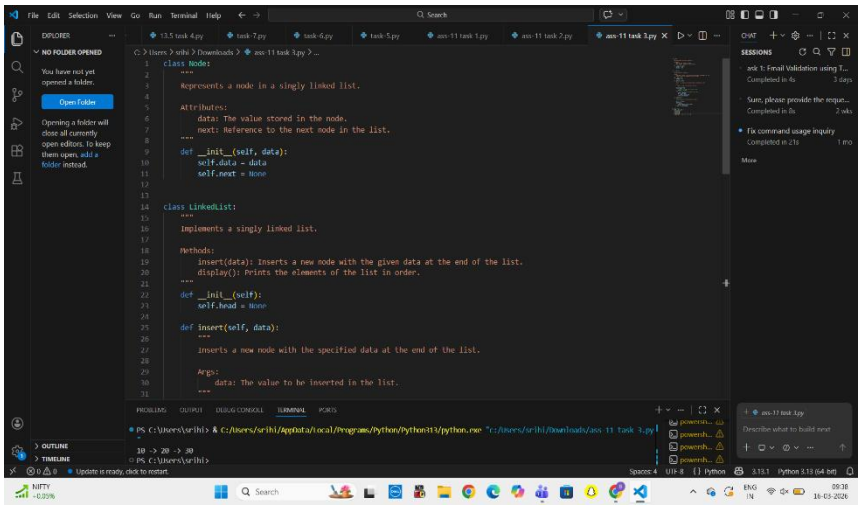
Sample Input Code:

```
class Node:
    pass

class LinkedList:
    pass
```

PROMPT : Generate Python code for a Singly Linked List.
Create a Node class and a LinkedList class.
The LinkedList should include methods to insert nodes and display the list.
Add clear comments and docstrings explaining how the linked list works.

CODE :



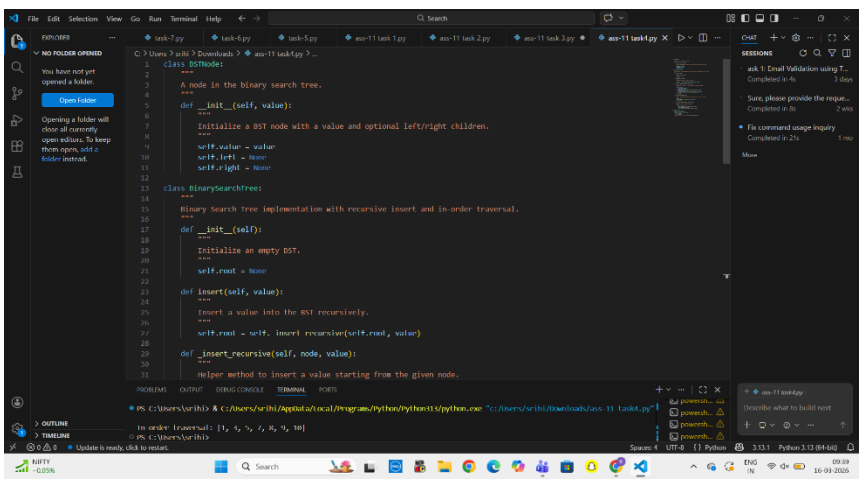
Expected Output:

- A working linked list implementation with clear method documentation.

Explanation : In this task, I implemented a Singly Linked List in Python. A linked list is a dynamic data structure where elements are connected

using nodes.
First, I created a Node class which stores the data and the reference to the next node. Then I created a LinkedList class which contains the head node.
I implemented an insert function to add new nodes to the list and a display function to print the elements of the linked list. After writing the program, I tested it by inserting multiple values and displaying the linked list.

Task Description #4 – Binary Search Tree (BST)
Task: Use AI to create a BST with insert and in-order traversal methods.
Sample Input Code:
class BST:
    pass
PROMPT : Write a Python program that implements a Binary Search Tree (BST).
Include methods for inserting nodes recursively and performing an in-order traversal.
Use a BST Node class for nodes and add comments and docstrings explaining the code.
CODE :



Expected Output:
- BST implementation with recursive insert and traversal methods.

Explanation : In this task, I implemented **a** Binary Search Tree (BST) in Python. A binary search tree is a hierarchical data structure where each node has at most two children.

In BST, values smaller than the root node are stored in the left subtree and values greater than the root node are stored in the right subtree.

I created a class called to represent each node in the tree. Then I implemented the method to add nodes and traversal to display the nodes in sorted order. Finally, I tested the program by inserting values into the tree.

Task Description #5 – Hash Table

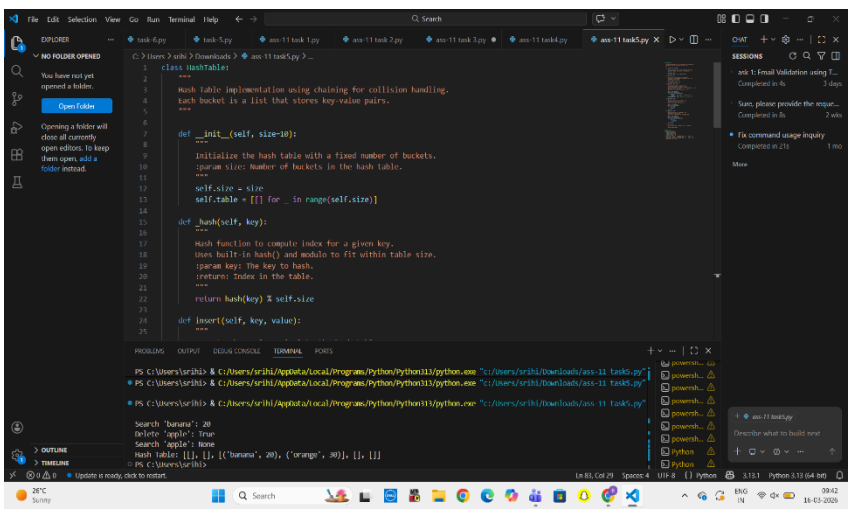
Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
    pass
```

PROMPT : Generate a Python implementation of a Hash Table with collision handling using chaining.
Include methods insert(), search(), and delete().
Use a hash function to determine the index and store elements in lists.
Add comments and docstrings explaining how hashing and collision handling work.

CODE :



Expected Output:

- Collision handling using chaining, with well-commented methods.

Explanation : In this task, I implemented a Hash Table using Python. A hash table stores data in key-value pairs and uses a hash function to map keys to specific indexes.

First, I created a class. I used a list of lists to store elements and handle collisions using chaining.

Then I implemented functions such as to add data, to find values using keys, and to remove elements. This structure helps in performing faster data retrieval.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

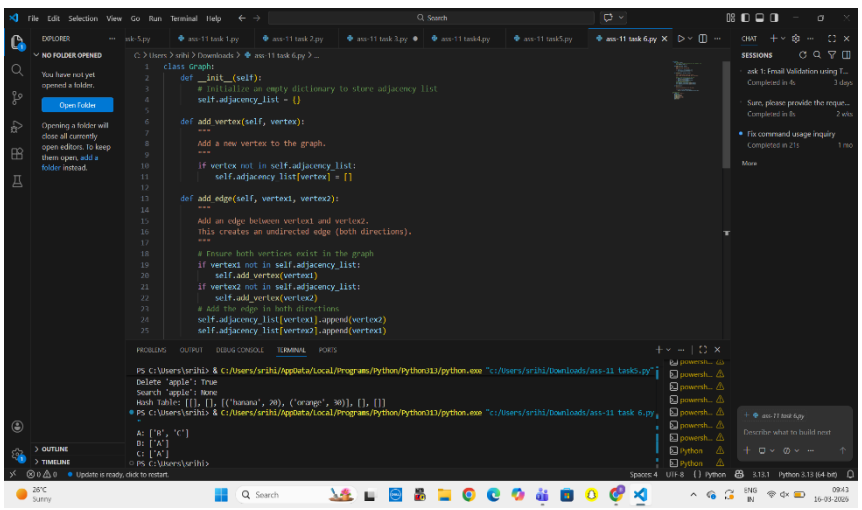
```
class Graph:
    pass
```

PROMPT : Write Python code to implement a graph using an adjacency list.

Create a Graph class with methods add_vertex(), add_edge(), and display().

Ensure the graph structure is easy to understand and include comments explaining the logic.

CODE :



Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Explanation : In this task, I implemented a Graph using an adjacency list representation**.**

A graph consists of vertices (nodes) and edges (connections). I created a class called and used a dictionary to store vertices and their connected nodes.

Then I implemented methods such as to add nodes and to connect two nodes. Finally, I displayed the graph structure to verify the implementation.

Task Description #7 – Priority Queue

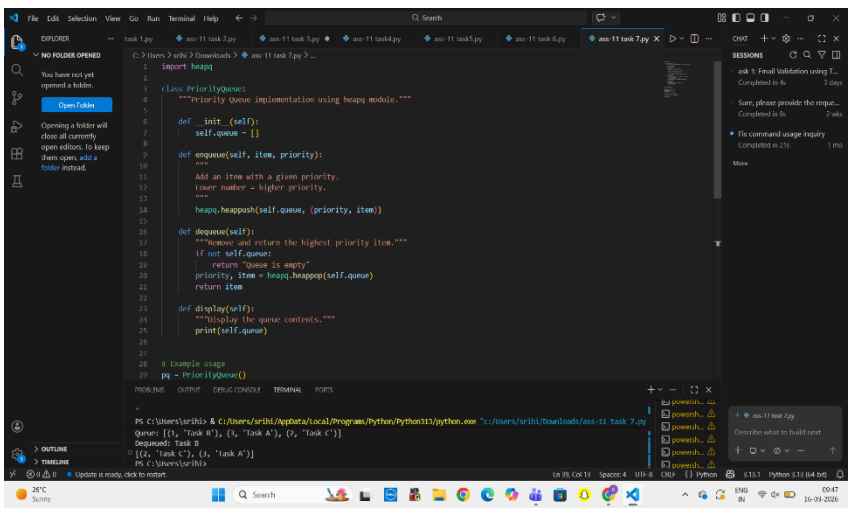
Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

PROMPT : Generate a Python class PriorityQueue using the heapq module.
Include methods enqueue(item, priority), dequeue(), and display().
Explain how the heap structure maintains priority order with comments and docstrings.

CODE :



Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Explanation : In this task, I implemented a Priority Queue using Python's heap queue module**.**

A priority queue processes elements based on priority instead of the order of insertion. Elements with higher priority are removed first.

I created a class and used the heap queue module to manage the heap structure. Then I implemented methods to insert elements with priority and remove the highest priority element.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

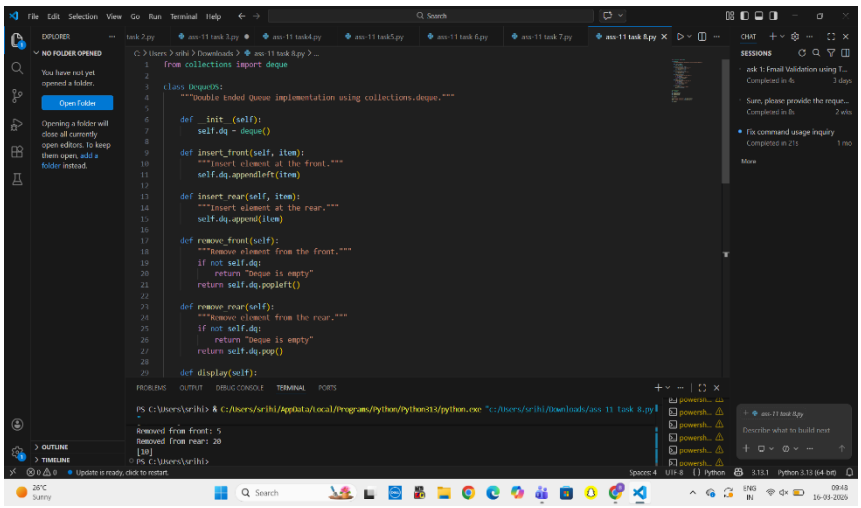
```
class DequeDS:
    pass
```

PROMPT : Write a Python class DequeDS that implements a double-ended queue using collections.deque.

Include methods to insert and remove elements from both the front and rear.

Add docstrings and comments explaining how a deque works.

CODE :



Expected Output:

- Insert and remove from both ends with docstrings.

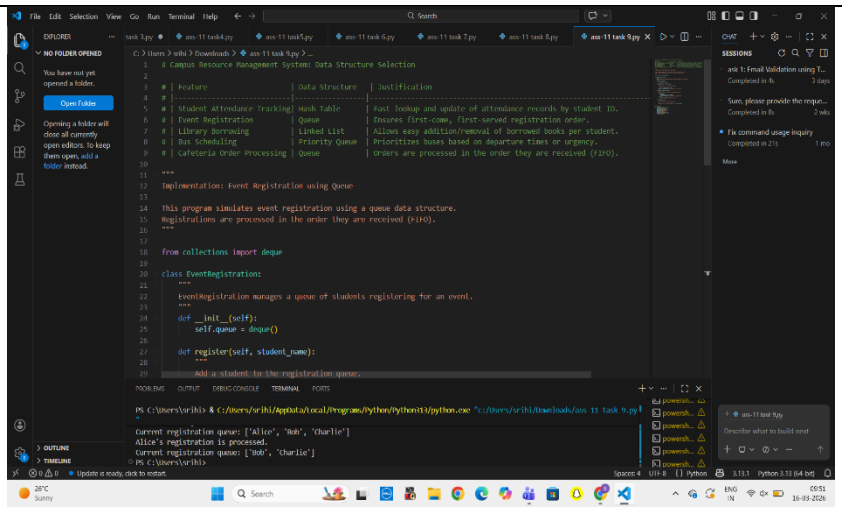
Explanation : In this task, I implemented a Deque (Double Ended Queue) in Python.

A deque allows insertion and deletion from both the front and rear ends. I used the module for this implementation.

Then I created functions to insert elements at the front and rear and also remove elements from both ends. Finally, I tested the deque operations.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

	Scenario:
	Your college wants to develop a Campus Resource Management System that handles:
	1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
	2. Event Registration System – Manage participants in events with quick search and removal.
	3. Library Book Borrowing – Keep track of available books and their due dates.
	4. Bus Scheduling System – Maintain bus routes and stop connections.
	5. Cafeteria Order Queue – Serve students in the order they arrive.
	Student Task:
	• For each feature, select the most appropriate data structure from the list below:
	  o Stack
	  o Queue
	  o Priority Queue
	  o Linked List
	  o Binary Search Tree (BST)
	  o Graph
	  o Hash Table
	  o Deque
	• Justify your choice in 2–3 sentences per feature.
	• Implement one selected feature as a working Python program with AI-assisted code generation.
	PROMPT : Given a campus resource management system with features like student attendance tracking,
	event registration, library borrowing, bus scheduling, and cafeteria order processing,
	select the most suitable data structure (Stack, Queue, Priority Queue, Linked List,
	Binary Search Tree, Graph, Hash Table, or Deque) for each feature.
	
	Provide a table mapping each feature to the selected data structure and give a 2–3 sentence justification for each choice.
	
	Then implement one selected feature as a Python program with proper comments and docstrings.
	
	CODE :	



Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Explanation : In this task, I analyzed different campus management features and selected appropriate data structures.

For example, queue was used for attendance tracking because students enter sequentially. Hash tables were used for event registration to allow fast searching of student IDs. Binary search trees were used for organizing library books, and graphs were used for bus route management.

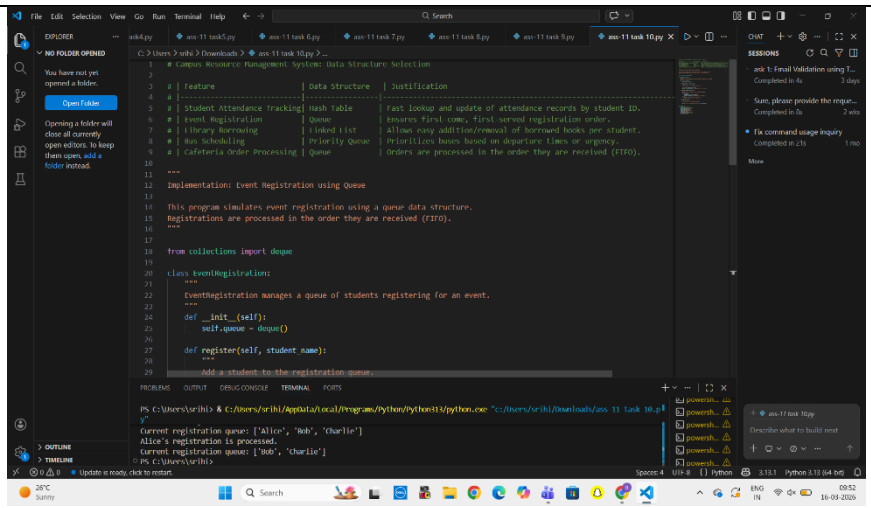
This task helped in understanding how data structures can be applied to real-world systems.

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.

	4. Product Search Engine – Fast lookup of products using product ID.
	5. Delivery Route Planning – Connect warehouses and delivery locations.
	
	Student Task:
	• For each feature, select the most appropriate data structure from the list below:
	  o Stack
	  o Queue
	  o Priority Queue
	  o Linked List
	  o Binary Search Tree (BST)
	  o Graph
	  o Hash Table
	  o Deque
	• Justify your choice in 2–3 sentences per feature.
	• Implement one selected feature as a working Python program with AI-assisted code generation.
	PROMPT : For a smart e-commerce platform with features such as shopping cart management,
	order processing, top-selling product tracking, product search, and delivery route planning,
	choose the most appropriate data structure from the following list: Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph, Hash Table, Deque.
	
	Create a table mapping feature → chosen data structure → justification (2–3 sentences).
	Also implement one selected feature as a Python program with proper documentation and comments.
	
	CODE :	



Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Explanation : In this task, I analyzed the features of a smart e-commerce platform and selected suitable data structures.

Linked lists were used for shopping cart management because items can be easily added or removed. Queues were used for order processing since orders must be processed in the order they are received. Priority queues were used for tracking top-selling products.

Hash tables were used for fast product searching, and graphs were used for delivery route planning. This analysis demonstrated how different data structures are useful in practical applications.